

# 使用 Cloud SQL Node.js 連接器，將完整堆疊 Angular 應用程式部署至 Cloud Run，並搭配 Cloud SQL for PostgreSQL

來源：<https://codelabs.developers.google.com/codelabs/deploy-application-with-database/cloud-sql-nodejs-connector-angular?hl=zh-tw>

步驟數：10

---

# 目錄

1. [1. 總覽](#)
2. [2. 必要條件](#)
3. [3. 專案設定](#)
4. [4. 開啟 Cloud Shell 編輯器](#)
5. [5. 啟用 API](#)
6. [6. 設定服務帳戶](#)
7. [7. 建立 Cloud SQL 資料庫](#)
8. [8. 準備應用程式](#)
9. [9. 將應用程式部署至 Cloud Run](#)
10. [10. 恭喜](#)

步驟 1 · 約 0 分鐘

## 1. 總覽

[Cloud Run](#) 是一個全代管平台，讓您能直接在可擴充的 Google 基礎架構上執行程式碼。本程式碼研究室將示範如何使用 Cloud SQL Node.js 連接器，將 Cloud Run 上的 Angular 應用程式連線至 PostgreSQL 適用的 [Cloud SQL](#) 資料庫。

本實驗室的學習內容包括：

- 建立 PostgreSQL 適用的 Cloud SQL 執行個體
  - 將應用程式部署至 Cloud Run，並連線至 Cloud SQL 資料庫
-

步驟 2 · 約 0 分鐘

## 2. 必要條件

1. 如果沒有 Google 帳戶，請先[建立帳戶](#)。
    - 請改用個人帳戶，而非公司或學校帳戶。公司和學校帳戶可能設有限制，導致您無法啟用本實驗室所需的 API。
-

步驟 3 · 約 0 分鐘

## 3. 專案設定

1. 登入 [Google Cloud 控制台](#)。
  2. 在 Cloud 控制台中[啟用帳單](#)。
    - 完成本實驗室的 Cloud 資源費用應不到 \$1 美元。
    - 您可以按照本實驗室結尾的步驟刪除資源，以免產生後續費用。
    - 新使用者可獲得價值 [\\$300 美元的免費試用期](#)。
  3. [建立新專案](#)，或選擇重複使用現有專案。
-

步驟 4 · 約 0 分鐘

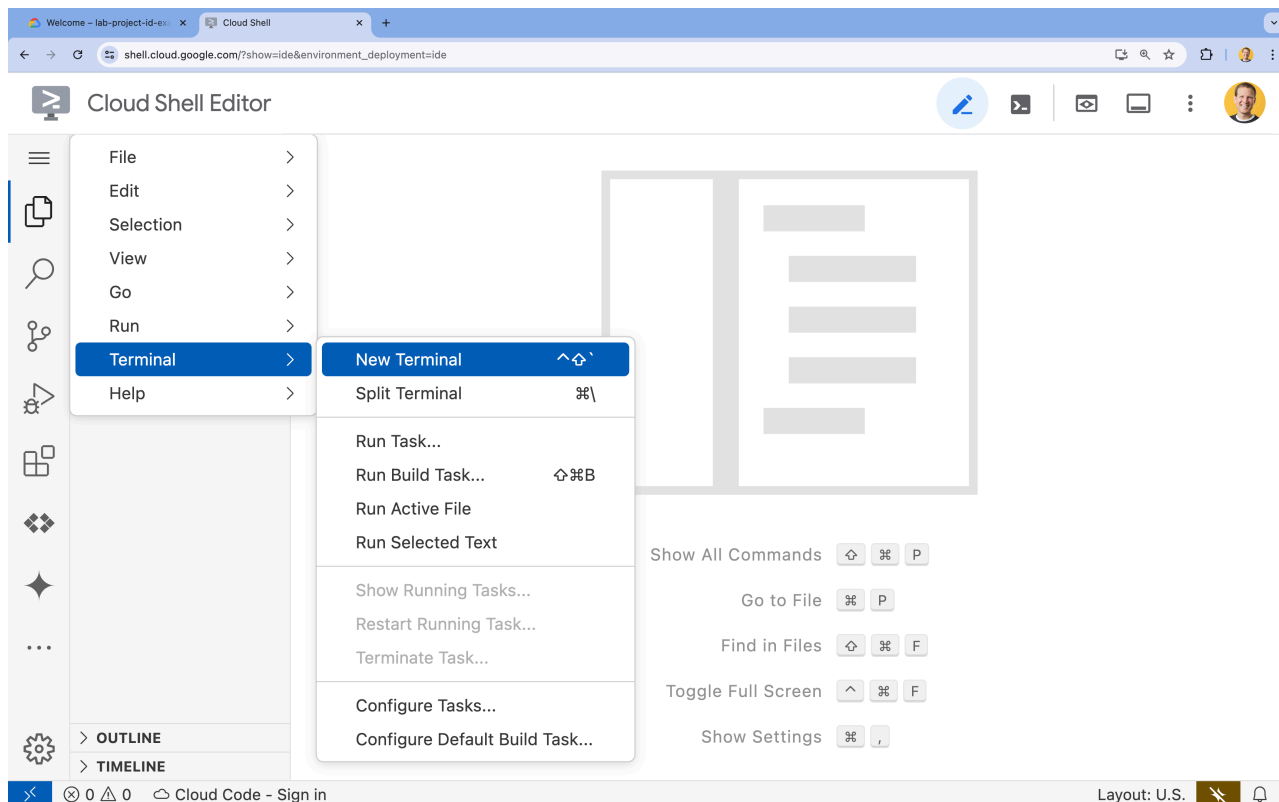
## 4. 開啟 Cloud Shell 編輯器

1. 前往 [Cloud Shell 編輯器](#)
2. 如果畫面底部未顯示終端機，請開啟終端機：

- 按一下漢堡選單



- 按一下「終端機」。
- 按一下「New Terminal」(新增終端機)



3. 在終端機中，使用下列指令設定專案：

- 格式：



```
gcloud config set project [PROJECT_ID]
```

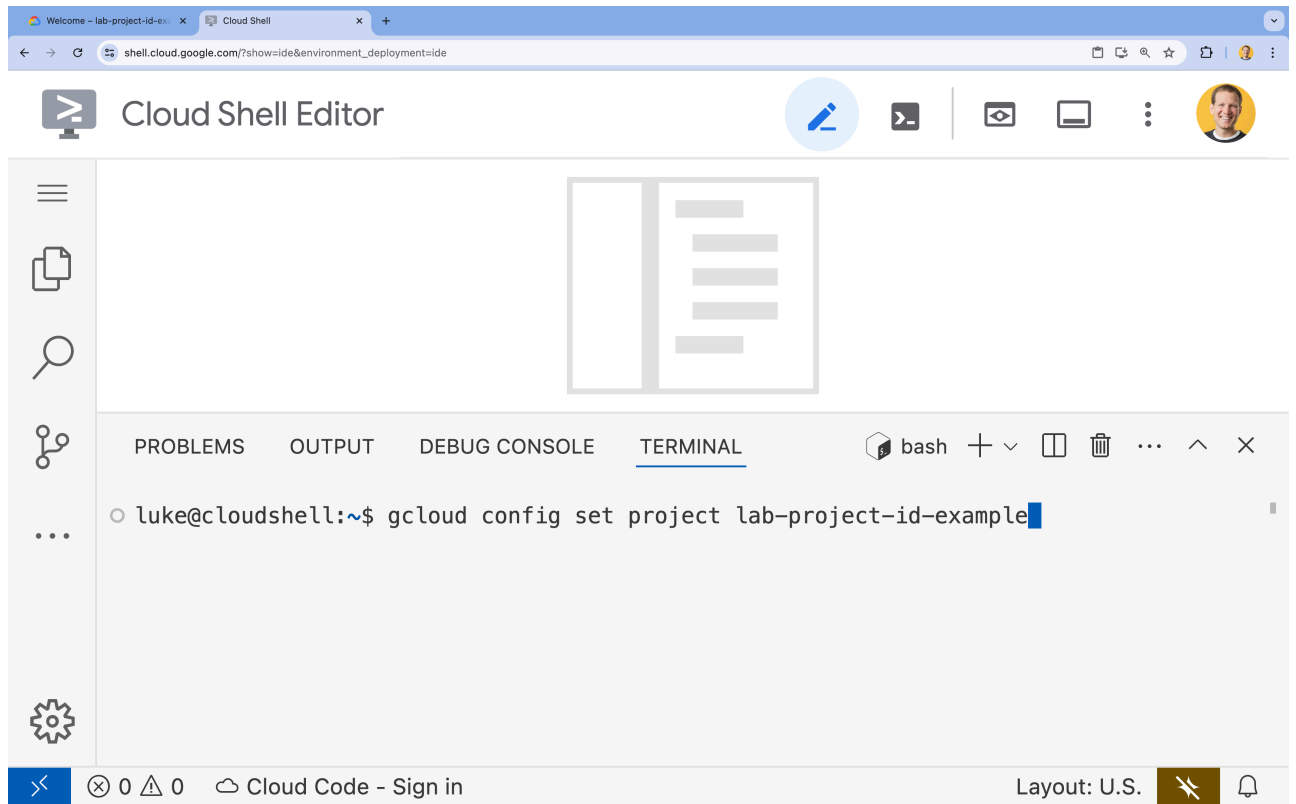
- 範例：

```
gcloud config set project lab-project-id-example
```

- 如果忘記專案 ID，請按照下列步驟操作：

- 您可以使用下列指令列出所有專案 ID：

```
gcloud projects list | awk '/PROJECT_ID/{print $2}'
```



4. 如果系統提示您授權，請點選「授權」繼續操作。

## Authorize Cloud Shell

gcloud is requesting your credentials to make a GCP API call.

Click to authorize this and future calls that require your credentials.

REJECT

AUTHORIZE

5. 您應會看到下列訊息：

```
Updated property [core/project].
```

如果看到 `WARNING` 並收到 `Do you want to continue (Y/N)?` 提示，可能是輸入的專案 ID 有誤。按下 `N` 和 `Enter`，然後再次嘗試執行 `gcloud config set project` 指令。

步驟 5 · 約 0 分鐘

## 5. 啟用 API

在終端機中啟用 API：

```
gcloud services enable \  
  sqladmin.googleapis.com \  
  run.googleapis.com \  
  artifactregistry.googleapis.com \  
  cloudbuild.googleapis.com
```

如果系統提示您授權，請點選「授權」繼續操作。

### Authorize Cloud Shell

gcloud is requesting your credentials to make a GCP API call.

Click to authorize this and future calls that require your credentials.

REJECT

AUTHORIZE

這個指令可能需要幾分鐘才能完成，但最終應該會產生類似以下的成功訊息：

```
Operation "operations/acf.p2-73d90d00-47ee-447a-b600" finished successfully.
```

---



## 6. 設定服務帳戶

建立及設定 Google Cloud 服務帳戶，供 Cloud Run 使用，確保該帳戶具備連線至 Cloud SQL 的正確權限。

1. 執行下列 `gcloud iam service-accounts create` 指令，建立新的服務帳戶：

```
gcloud iam service-accounts create quickstart-service-account \
  --display-name="Quickstart Service Account"
```

2. 執行下列 `gcloud projects add-iam-policy-binding` 指令，將「Cloud SQL Client」(Cloud SQL 用戶端) 角色新增至您剛才建立的 Google Cloud 服務帳戶。

```
gcloud projects add-iam-policy-binding ${GOOGLE_CLOUD_PROJECT} \
  --member="serviceAccount:quickstart-service-
account@${GOOGLE_CLOUD_PROJECT}.iam.gserviceaccount.com" \
  --role="roles/cloudsql.client"
```

3. 執行下列 `gcloud projects add-iam-policy-binding` 指令，將「Cloud SQL Instance User」(Cloud SQL 執行個體使用者) 角色新增至您剛建立的 Google Cloud 服務帳戶。

```
gcloud projects add-iam-policy-binding ${GOOGLE_CLOUD_PROJECT} \
  --member="serviceAccount:quickstart-service-
account@${GOOGLE_CLOUD_PROJECT}.iam.gserviceaccount.com" \
  --role="roles/cloudsql.instanceUser"
```

4. 執行下列 `gcloud projects add-iam-policy-binding` 指令，將「記錄檔寫入者」角色新增至您剛建立的 Google Cloud 服務帳戶。

```
gcloud projects add-iam-policy-binding ${GOOGLE_CLOUD_PROJECT} \
  --member="serviceAccount:quickstart-service-
account@${GOOGLE_CLOUD_PROJECT}.iam.gserviceaccount.com" \
  --role="roles/logging.logWriter"
```

---

步驟 7 · 約 0 分鐘

## 7. 建立 Cloud SQL 資料庫

1. 執行 `gcloud sql instances create` 指令，建立 Cloud SQL 執行個體

```
gcloud sql instances create quickstart-instance \  
  --database-version=POSTGRES_14 \  
  --cpu=4 \  
  --memory=16GB \  
  --region=us-central1 \  
  --database-flags=cloudsql.iam_authentication=on
```

這個指令可能需要幾分鐘才能完成。

1. 執行 `gcloud sql databases create` 指令，在 `quickstart-instance` 中建立 Cloud SQL 資料庫。

```
gcloud sql databases create quickstart_db \  
  --instance=quickstart-instance
```

2. 為先前建立的服務帳戶建立 PostgreSQL 資料庫使用者，以便存取資料庫。

```
gcloud sql users create quickstart-service-account@${GOOGLE_CLOUD_PROJECT}.iam \  
  --instance=quickstart-instance \  
  --type=cloud_iam_service_account
```

---

步驟 8 · 約 0 分鐘

## 8. 準備應用程式

準備回應 HTTP 要求的 Next.js 應用程式。

1. 如要建立名為 `task-app` 的新 Next.js 專案，請使用下列指令：

```
npx --yes @angular/cli@19.2.5 new task-app \
  --minimal \
  --inline-template \
  --inline-style \
  --ssr \
  --server-routing \
  --defaults
```

2. 將目錄變更為 `task-app`：

```
cd task-app
```

1. 安裝 `pg` 和 Cloud SQL Node.js 連接器程式庫，與 PostgreSQL 資料庫互動。

```
npm install pg @google-cloud/cloud-sql-connector google-auth-library
```

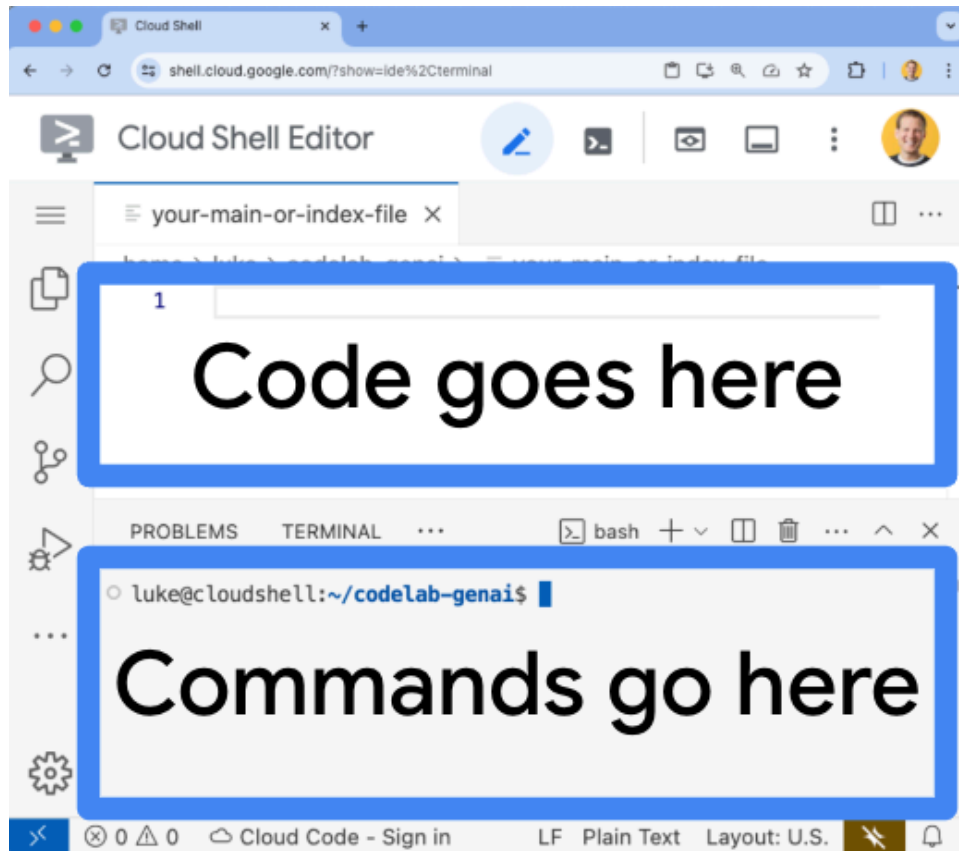
2. 安裝 `@types/pg` 做為開發依附元件，即可使用 TypeScript Next.js 應用程式。

```
npm install --save-dev @types/pg
```

1. 在 Cloud Shell 編輯器中開啟 `server.ts` 檔案：

```
cloudshell edit src/server.ts
```

畫面頂端現在應該會顯示檔案。您可以在這裡編輯這個 `server.ts` 檔案。



2. 刪除 `server.ts` 檔案的現有內容。
3. 複製下列程式碼，並貼到開啟的 `server.ts` 檔案中：

```

import {
  AngularNodeAppEngine,
  createNodeRequestHandler,
  isMainModule,
  writeResponseToNodeResponse,
} from '@angular/ssr/node';
import express from 'express';
import { dirname, resolve } from 'node:path';
import { fileURLToPath } from 'node:url';
import pg from 'pg';
import { AuthTypes, Connector } from '@google-cloud/cloud-sql-connector';
import { GoogleAuth } from 'google-auth-library';
const auth = new GoogleAuth();

const { Pool } = pg;

type Task = {
  id: string;
  title: string;
  status: 'IN_PROGRESS' | 'COMPLETE';
  createdAt: number;
};

const projectId = await auth.getProjectId();

const connector = new Connector();
const clientOpts = await connector.getOptions({
  instanceConnectionName: `${projectId}:us-central1:quickstart-instance`,
  authType: AuthTypes.IAM,
});

const pool = new Pool({
  ...clientOpts,
  user: `quickstart-service-account@${projectId}.iam`,
  database: 'quickstart_db',
});

const tableCreationIfDoesNotExist = async () => {
  await pool.query(`CREATE TABLE IF NOT EXISTS tasks (
    id SERIAL NOT NULL,
    created_at timestamp NOT NULL,
    status VARCHAR(255) NOT NULL default 'IN_PROGRESS',
    title VARCHAR(1024) NOT NULL,
    PRIMARY KEY (id)
  );`);
}

const serverDistFolder = dirname(fileURLToPath(import.meta.url));
const browserDistFolder = resolve(serverDistFolder, '../browser');

const app = express();
const angularApp = new AngularNodeAppEngine();

```

```

app.use(express.json());

app.get('/api/tasks', async (req, res) => {
  await tableCreationIfDoesNotExist();
  const { rows } = await pool.query(`SELECT id, created_at, status, title FROM tasks
ORDER BY created_at DESC LIMIT 100`);
  res.send(rows);
});

app.post('/api/tasks', async (req, res) => {
  const newTaskTitle = req.body.title;
  if (!newTaskTitle) {
    res.status(400).send("Title is required");
    return;
  }
  await tableCreationIfDoesNotExist();
  await pool.query(`INSERT INTO tasks(created_at, status, title) VALUES(NOW(),
'IN_PROGRESS', $1)`, [newTaskTitle]);
  res.sendStatus(200);
});

app.put('/api/tasks', async (req, res) => {
  const task: Task = req.body;
  if (!task || !task.id || !task.title || !task.status) {
    res.status(400).send("Invalid task data");
    return;
  }
  await tableCreationIfDoesNotExist();
  await pool.query(
    `UPDATE tasks SET status = $1, title = $2 WHERE id = $3`,
    [task.status, task.title, task.id]
  );
  res.sendStatus(200);
});

app.delete('/api/tasks', async (req, res) => {
  const task: Task = req.body;
  if (!task || !task.id) {
    res.status(400).send("Task ID is required");
    return;
  }
  await tableCreationIfDoesNotExist();
  await pool.query(`DELETE FROM tasks WHERE id = $1`, [task.id]);
  res.sendStatus(200);
});

/**
 * Serve static files from /browser
 */
app.use(
  express.static(browserDistFolder, {

```

```

    maxAge: '1y',
    index: false,
    redirect: false,
  )),
);

/**
 * Handle all other requests by rendering the Angular application.
 */
app.use('/**', (req, res, next) => {
  angularApp
    .handle(req)
    .then((response) =>
      response ? writeResponseToNodeResponse(response, res) : next(),
    )
    .catch(next);
});

/**
 * Start the server if this module is the main entry point.
 * The server listens on the port defined by the `PORT` environment variable, or defaults
 * to 4000.
 */
if (isMainModule(import.meta.url)) {
  const port = process.env['PORT'] || 4000;
  app.listen(port, () => {
    console.log(`Node Express server listening on http://localhost:${port}`);
  });
}

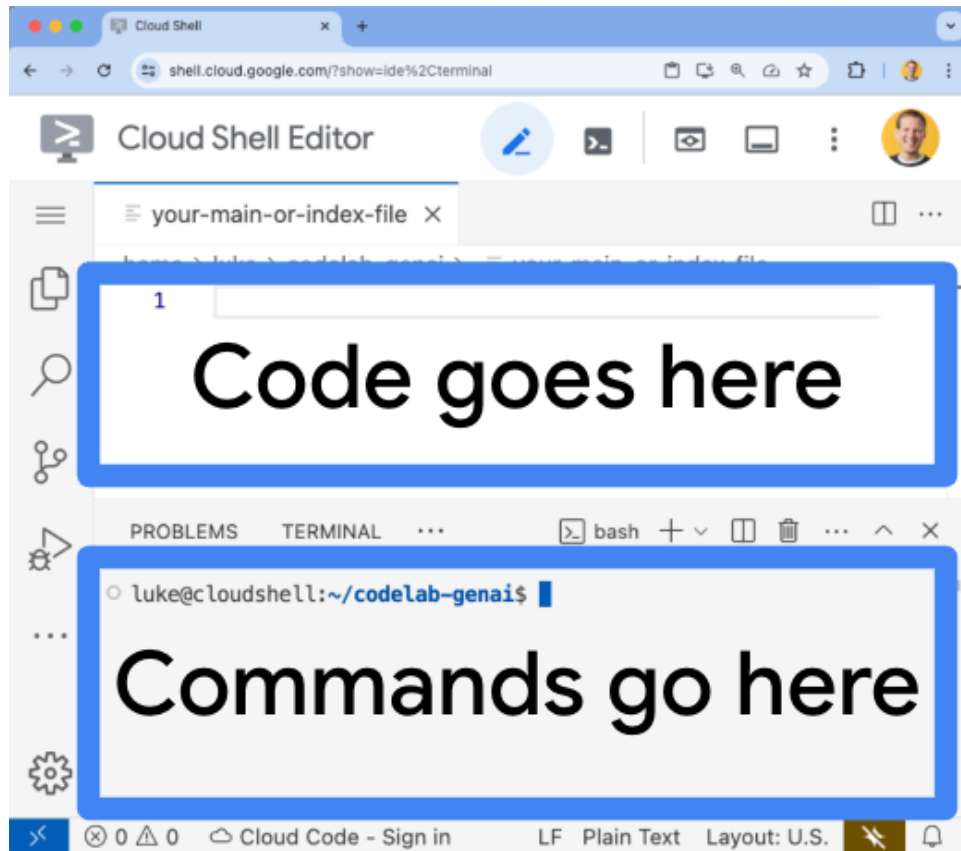
/**
 * Request handler used by the Angular CLI (for dev-server and during build) or Firebase
 * Cloud Functions.
 */
export const reqHandler = createNodeRequestHandler(app);

```

1. 在 Cloud Shell 編輯器中開啟 `app.component.ts` 檔案：

```
cloudshell edit src/app/app.component.ts
```

畫面頂端現在應該會顯示現有檔案。您可以在這裡編輯這個 `app.component.ts` 檔案。



2. 刪除 `app.component.ts` 檔案的現有內容。
3. 複製下列程式碼，並貼到開啟的 `app.component.ts` 檔案中：



```

import { afterNextRender, Component, signal } from '@angular/core';
import { FormsModule } from '@angular/forms';

type Task = {
  id: string;
  title: string;
  status: 'IN_PROGRESS' | 'COMPLETE';
  createdAt: number;
};

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [FormsModule],
  template: `
    <section>
      <input
        type="text"
        placeholder="New Task Title"
        [(ngModel)]="newTaskTitle"
        class="text-black border-2 p-2 m-2 rounded"
      />
      <button (click)="addTask()">Add new task</button>
      <table>
        <tbody>
          @for (task of tasks(); track task) {
            @let isComplete = task.status === 'COMPLETE';
            <tr>
              <td>
                <input
                  (click)="updateTask(task, { status: isComplete ? 'IN_PROGRESS' :
'COMPLETE' })"
                  type="checkbox"
                  [checked]="isComplete"
                />
              </td>
              <td>{{ task.title }}</td>
              <td>{{ task.status }}</td>
              <td>
                <button (click)="deleteTask(task)">Delete</button>
              </td>
            </tr>
          }
        </tbody>
      </table>
    </section>
  `,
  styles: '',
})
export class AppComponent {
  newTaskTitle = '';
  tasks = signal<Task[]>([]);

```

```

constructor() {
  afterNextRender({
    earlyRead: () => this.getTasks()
  });
}

async getTasks() {
  const response = await fetch(`/api/tasks`);
  const tasks = await response.json();
  this.tasks.set(tasks);
}

async addTask() {
  await fetch(`/api/tasks`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      title: this.newTaskTitle,
      status: 'IN_PROGRESS',
      createdAt: Date.now(),
    }),
  });
  this.newTaskTitle = '';
  await this.getTasks();
}

async updateTask(task: Task, newTaskValues: Partial<Task>) {
  await fetch(`/api/tasks`, {
    method: 'PUT',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ ...task, ...newTaskValues }),
  });
  await this.getTasks();
}

async deleteTask(task: any) {
  await fetch('/api/tasks', {
    method: 'DELETE',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(task),
  });
  await this.getTasks();
}
}

```

應用程式現在已可部署。

步驟 9 · 約 0 分鐘

## 9. 將應用程式部署至 Cloud Run

1. 執行下列指令，將應用程式部署至 Cloud Run：

```
gcloud run deploy to-do-tracker \
  --region=us-central1 \
  --source=. \
  --service-account="quickstart-service-
account@${GOOGLE_CLOUD_PROJECT}.iam.gserviceaccount.com" \
  --allow-unauthenticated
```

2. 如果系統提示您確認是否要繼續，請按下 **Y** 和 **Enter**：

```
Do you want to continue (Y/n)? Y
```

幾分鐘後，應用程式應會提供網址供您造訪。

前往該網址，即可查看應用程式的運作情形。每次造訪網址或重新整理頁面時，都會看到工作應用程式。

---

## 10. 恭喜

在本實驗室中，您已學會如何執行下列工作：

- 建立 PostgreSQL 適用的 Cloud SQL 執行個體
- 將應用程式部署至 Cloud Run，並連線至 Cloud SQL 資料庫

## 清除所用資源

Cloud SQL 沒有免費方案，如果您繼續使用，系統會向您收費。您可以刪除 Cloud 專案，以免產生額外費用。

不使用服務時，Cloud Run 不會收費，但您可能仍須支付容器映像檔在 Artifact Registry 的儲存費用。刪除 Cloud 專案後，系統就會停止對專案使用的所有資源收取費用。

如要刪除專案，請按照下列步驟操作：

```
gcloud projects delete $GOOGLE_CLOUD_PROJECT
```

您也可以從 Cloud Shell 磁碟刪除不必要的資源。您可以：

1. 刪除 Codelab 專案目錄：

```
rm -rf ~/task-app
```

2. 警告！這項操作無法復原，如要刪除 Cloud Shell 中的所有內容來釋出空間，可以[刪除整個主目錄](#)。請務必將要保留的內容另存到其他位置。

```
sudo rm -rf $HOME
```

## 持續掌握情況

- 使用 [Cloud SQL Node.js 連接器](#)，將全端 Next.js 應用程式部署至 Cloud Run，並搭配 PostgreSQL 適用的 Cloud SQL
  - 使用 [Cloud SQL Node.js 連接器](#)，將全端 Angular 應用程式部署至 Cloud Run，並搭配 PostgreSQL 適用的 Cloud SQL
  - 使用 [Node.js Admin SDK](#)，將全端 Angular 應用程式部署至 Cloud Run，並搭配使用 Firestore
  - 使用 [Node.js Admin SDK](#)，將全端 Next.js 應用程式部署至 Cloud Run，並搭配 Firestore 使用
-